MLT- Lab 4

Title:

Naive Bayes Classifier Implementation and Comparative Analysis: Algorithm Development, Variant Evaluation, and Performance Benchmarking Against Tree-Based Models

Dataset Descriptiom:

Dataset 1: Categorical Dataset (Tennis Play Prediction)

Size & Structure: 1000 samples, 4 features (columns), 2 target classes.

Attributes: All 4 columns are categorical variables named 'Weather', 'Temperature', 'Humidity', and 'Wind', representing environmental conditions for tennis play prediction.

Data Types: All features are categorical with the following categories:

- Weather: {Sunny, Rainy, Cloudy, Snowy}
- Temperature: {Hot, Warm, Cool, Cold}
- Humidity: {High, Medium, Low}
- Wind: {Strong, Moderate, Weak}

Distribution: The features are randomly generated with uniform distribution across categories. The target variable 'PlayTennis' is derived using rule-based logic with added randomness, creating two classes: 'Yes' and 'No'. Class distribution is approximately balanced.

Train-Test Split: 70% training data (700 samples) and 30% testing data (300 samples), random stratified split.

Dataset 2: Mixed Dataset (Loan Approval Prediction)

Size & Structure: 1000 samples, 5 features (columns), 2 target classes.

Attributes:

- 3 Numerical features: Age (18-70 years), Income ($30K-$90K), CreditScore (300-850)
- 2 Categorical features: Education {High School, Bachelor, Master, PhD}, Employment {Employed, Unemployed, Self-Employed}

Data Types: Mixed - 3 float, 64 numerical features and 2 categorical features.

Distribution: Numerical features follow normal and uniform distributions. The target variable 'LoanStatus' is generated using multi-criteria decision logic with randomness, creating two classes: 'Approved' and 'Rejected'. Class distribution is approximately balanced (55-45 split).

Train-Test Split: 70% training data (700 samples) and 30% testing data (300 samples), random stratified split.

Pseudo code:

Algorithm: Naive Bayes & Comparative Analysis

INPUT: X_categorical, y_categorical, X_mixed, y_mixed

STEP 1: Custom Hybrid Naive Bayes Implementation

```
Python
CLASS NaiveBayesClassifier:
   FUNCTION fit(X, y):
      P(c) = count(class) / total
      FOR each feature f:
         IF numerical: Store mean(f|c), std(f|c)
         ELSE: Store P(f=val|c) (Categorical)

   FUNCTION predict(X):
      FOR each sample x:
         score(c) = log(P(c)) + SUM(log(P(feature_val|c)))
         predicted = argmax(score(c))
      RETURN predictions
```

STEP 2: Data Preprocessing

1. **Categorical:** LabelEncode all columns → Split (70/30).
2. **Mixed:** LabelEncode categories + StandardScale numericals → Split (70/30).

STEP 3: Integrated Model Experiments

Run the following suite on **both** datasets (Categorical & Mixed):

Category	**Models/Variations**
Naive Bayes	Custom NB, GaussianNB (Default & Tuned 10−8), CategoricalNB (Laplace 1.0)
Tree-Based	Decision Tree (Depth 5 vs 10), Random Forest (Est. 100 vs 200)

For each model:

- **Train/Test:** Fit on Training set, Predict on Test set.
- **CV:** Perform 5-fold Cross-Validation for stability.

STEP 4: Evaluation & Visualization

1. **Metrics:** Calculate Accuracy, Weighted Precision, Recall, and F1-Score for all variations.
2. **Tabulate:** Generate results_table_categorical and results_table_mixed.
3. **Visualize: * Bar Charts:** Compare Accuracy across all models for both datasets.
    - **Grouped Bars:** Plot multi-metrics (P, R, F1) to identify bias/variance.

STEP 5: Final Analysis

- Compare **Custom NB** vs. **Scikit-Learn** implementations.
- Evaluate the performance gap between **Naive Bayes** (Probabilistic) and **Random Forests** (Ensemble).
- **Recommendation:** Identify the optimal model based on the Accuracy/Complexity trade-off.

OUTPUT: 2 Result Tables, 4 Comparative Plots, Performance Analysis Report.

END

Result analysis:

ACCURACY COMPARISON TABLE

Model	Accuracy	Precision	Recall	F1-Score	CV Mean	CV Std
Custom NB (Scratch)	0.590	0.592	0.590	0.587	-	-
Gaussian NB	0.593	0.595	0.593	0.590	0.601	0.028
Gaussian NB (Tuned)	0.593	0.595	0.593	0.590	0.601	0.028
Categorical NB	0.590	0.592	0.590	0.587	0.599	0.026
Categorical NB (α=1.0)	0.590	0.592	0.590	0.587	0.599	0.026
Decision Tree	0.573	0.576	0.573	0.571	0.579	0.024
Decision Tree (Tuned)	0.587	0.589	0.587	0.585	0.586	0.021
Random Forest	0.603	0.604	0.603	0.602	0.607	0.019
Random Forest (Tuned)	**0.610**	**0.611**	**0.610**	**0.609**	**0.611**	**0.018**

ACCURACY COMPARISON TABLE - MIXED DATASET

Model	Accuracy	Precision	Recall	F1-Score	CV Mean	CV Std
Custom NB (Scratch)	0.740	0.740	0.740	0.740	-	-
Gaussian NB	0.740	0.740	0.740	0.740	0.741	0.019
Gaussian NB (Scaled)	0.740	0.740	0.740	0.740	0.741	0.019
Gaussian NB (Tuned)	0.740	0.740	0.740	0.740	0.741	0.019
Decision Tree	0.723	0.724	0.723	0.723	0.729	0.021
Decision Tree (Tuned)	0.760	0.761	0.760	0.760	0.754	0.016
Random Forest	0.783	0.784	0.783	0.783	0.781	0.013
Random Forest (Tuned)	**0.800**	**0.801**	**0.800**	**0.800**	**0.795**	**0.011**

CONSISTENCY ANALYSIS

Dataset	Method	Mean Accuracy	Std Dev	Stability
Categorical	Naive Bayes Variants	0.591	0.002	High
Categorical	Decision Tree	0.580	0.010	Moderate
Categorical	Random Forest	0.607	0.005	High
Mixed	Naive Bayes Variants	0.740	0.000	Perfect
Mixed	Decision Tree	0.742	0.026	Moderate
Mixed	Random Forest	0.792	0.012	High

COMPARISON SUMMARY

Dataset	Best NB Variant	NB Accuracy	Best Overall	Overall Accuracy	Performance Gap
Categorical	Gaussian NB	0.593	Random Forest (Tuned)	0.610	+2.9%
Mixed	Gaussian NB	0.740	Random Forest (Tuned)	0.800	+8.1%

Observation:

Random Forest (Tuned) achieved the highest accuracy on both datasets (61.0% categorical, 80.0% mixed), outperforming Naive Bayes by 2.9% and 8.1% respectively. The custom Naive Bayes implementation validated the algorithm with near-identical performance to sklearn (59.0-59.3% categorical, 74.0% mixed). Hyperparameter tuning and feature scaling had negligible impact on Naive Bayes (<0.1% change), while NB exhibited superior consistency with perfect stability on mixed data (std=0.000) compared to Random Forest (std=0.005-0.012) and Decision Trees (std=0.010-0.026). **The optimal model selection is Random Forest (Tuned) for maximum accuracy or Gaussian Naive Bayes for production systems prioritizing speed, consistency, and interpretability over marginal accuracy gains.**

Additional Data:



